

- First start building Datapath and Controller Path module in Verilog/System Verilog
- Define Instruction memory and fill the machine codes in it.
- Then, define Data memory
- Start instantiating controller module and Datapath module in “ riscvingle” module .
- In the TOP level module, instantiate riscvsingle, Instruction memory and Data memory modules as shown in figure 14.
- Finally develop Testbench to test DUT( Top level module) and inspect the waveform and performance of CPU.

### 2.2.1 Pipelined Processor

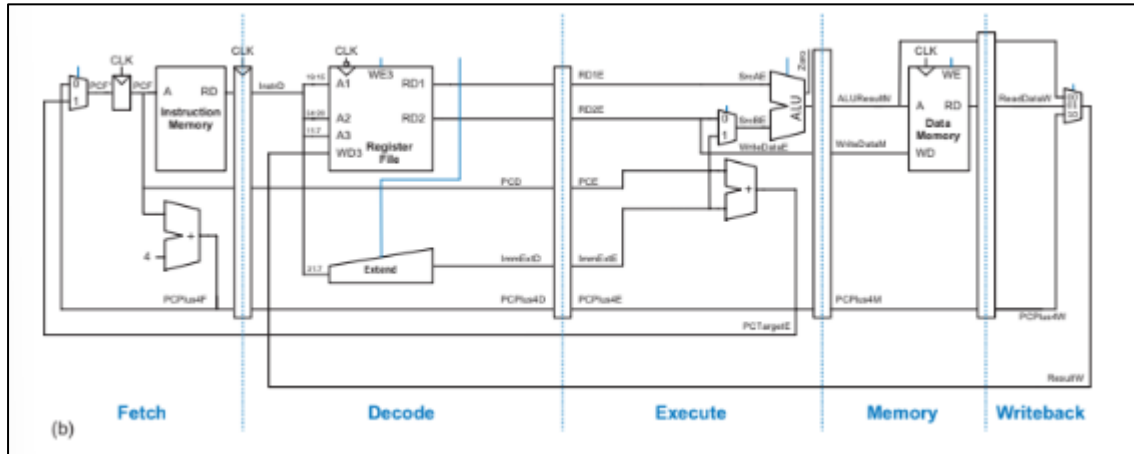
We design a pipelined processor by subdividing the single-cycle processor into five pipeline stages. Thus, five instructions can execute simultaneously, one in each stage. Because each stage has only one-fifth of the entire logic, the clock frequency is approximately five times faster. So, ideally, the latency of each instruction is unchanged, but the throughput is five times better. Microprocessors execute millions or billions of instructions per second, so throughput is more important than latency. Pipelining introduces some overhead, so the throughput will not be as high as we might ideally desire, but pipelining nevertheless gives such great advantage for so little cost that all modern high-performance microprocessors are pipelined.

Reading and writing the memory and register file and using the ALU typically constitute the biggest delays in the processor. We choose five pipeline stages so that each stage involves exactly one of these slow steps. Specifically, we call the five stages Fetch, Decode, Execute, Memory, and Writeback. They are similar to the five steps that the multicycle processor used to perform lw. In the Fetch stage, the processor reads the instruction from instruction memory. In the Decode stage, the processor reads source operands from the register file and decodes the instruction to produce the control signals. In the Execute stage, the processor performs a computation with the ALU. In the Memory stage, the processor reads or writes data memory, if applicable. Finally, in the Writeback stage, the processor writes the result to the register file, if applicable.

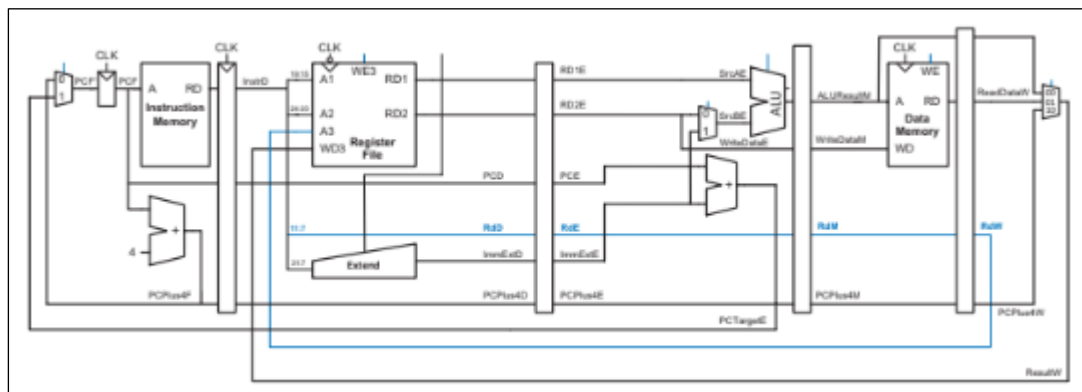
#### Pipeline Datapath:

The pipelined datapath is formed by chopping the single-cycle datapath into five stages separated by pipeline registers. Figure 15 shows the pipelined datapath formed by inserting four pipeline registers to separate the datapath into five stages. The stages and their boundaries are indicated in blue. Signals are given a suffix (F, D, E, M, or W) to indicate the stage in which they reside. The register file is peculiar because it is read in the Decode stage and written in the Writeback stage. So, although the register file is drawn in the Decode stage, its write address and write data come from the Writeback stage. This feedback will lead to pipeline hazards. The register file in the pipelined processor writes on the falling edge of CLK so that it can write a result in the first half of a cycle and read that result in the second half of the cycle for use in a subsequent instruction. One of the subtle but critical issues in pipelining is that all signals associated with a particular instruction must advance through the pipeline in unison. Figure 15 has an error related to this issue. Can you find it? The error is in the register file write logic, which should operate in the Writeback stage. The data value comes from ResultW, a Writeback

stage signal. But the destination register comes from RdD (InstrD11:7), which is a Decode stage signal. In the pipeline diagram of Figure 7.48, during cycle 5, the result of the lw instruction would be incorrectly written to s5 rather than s2.



**Figure 15 Pipelined Datapath with Cycles**



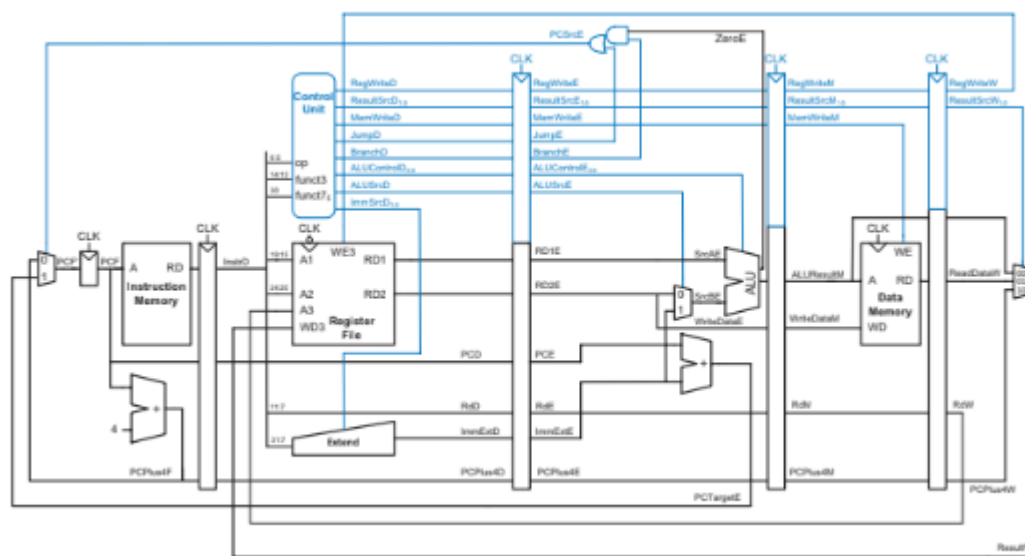
**Figure 16 Pipeline Corrected Datapath**

Figure 16 shows a corrected datapath, with the modification in blue. The Rd signal is now pipelined along through the Execution, Memory, and Writeback stages, so it remains in sync with the rest of the instruction. RdW and ResultW are fed back together to the register file in the Writeback stage. The astute reader may note that the logic to produce PCF' (the nextPC) is also problematic because it could be updated with either a Fetch or an Execute stage signal (PCPlus4F or PCTargetE).

## Pipelined Control

The pipelined processor uses the same control signals as the single-cycle processor and, therefore, has the same control unit. The control unit examines the `op`, `funct3`, and `funct7` fields of the instruction in the Decode stage to produce the control signals for the single-cycle processor. These control signals must be pipelined along with the data so that they remain synchronized with the instruction.

The entire pipelined processor with control is shown in Figure 17. RegWrite must be pipelined into the Writeback stage before it feeds back to the register file, just as Rd was pipelined in Figure 7.50. In addition to R-type ALU instructions, lw, sw, and beq, this pipelined processor also supports jal and I-type ALU instructions.



### Figure 17 Pipelined Control Unit

### Pipeline Hazard unit:

In a pipelined system, multiple instructions are handled concurrently. When one instruction is dependent on the results of another that has not yet completed, a hazard occurs. The register file is written during the first half of the cycle and read during the second half of the cycle, so a register can be written and read back in the same cycle without introducing a hazard.

### 1) Data Hazard Problem and its Solution

### Problem:

|     |            |                     |
|-----|------------|---------------------|
| Add | s8, s1, s2 | # s8 = s1 + s2      |
| sub | t0, s8, s3 | # uses s8 too early |
| or  | t1, s8, s4 | # also too early    |
| and | t2, s8, s5 | # OK (late enough)  |

1<sup>st</sup> instruction computes result early but writes to register late. Subsequent instruction read old value of s8. This is RAW(read after write) Data Hazard.

### Solution 1: Software

|            |            |
|------------|------------|
| add        | s8, s1, s2 |
| <b>nop</b> |            |
| <b>nop</b> |            |
| sub        | t0, s8, s3 |
| or         | t1, s8, s4 |

### Solution 2: Hardware (forwarding)

|     |                                           |
|-----|-------------------------------------------|
| add | s8, s1, s2                                |
| sub | t0, s8, s3 # value forwarded from ALU/MEM |
| or  | t1, s8, s4 # forwarded from WB stage      |
| and | t2, s8, s5 # normal read (no hazard)      |

| Software (NOP)                                                         | Hardware (forwarding)                                                                           |
|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Insert nop (no-operation) instructions between dependent instructions. | Instead of waiting for register write: Send result directly from pipeline stages to ALU inputs. |
| Delays dependent instructions until data is ready.                     | No Delay, Eliminates Most RAW Hazard                                                            |
| Wastes Cycles                                                          | Value forwarded directly, not read from register file.                                          |
| Burdens Compiler                                                       |                                                                                                 |

Forwarding happens when Source registers in Execution Stage matches:

- Destination in Memory Stage
- Destination in WriteBack Stage
- WRITE signal is Enabled

If both the Memory and Writeback stages contain matching destination registers, then the Memory stage should have priority because it contains the more recently executed instruction.

| Forward from    | Forward to      | Condition                 | forwardAE value |
|-----------------|-----------------|---------------------------|-----------------|
| Memory Stage    | Execution Stage | (Rs1E == RdM) & RegwriteM | 2'b10           |
| WriteBack Stage | Execution Stage | (Rs1E == RdW) & RegWriteW | 2'b01           |
| Forward from    | Forward to      | Condition                 | forwardBE value |
| Memory Stage    | Ex Stage        | (Rs2E == RdM) & RegwriteM | 2'b10           |
| WriteBack Stage | Ex Stage        | (Rs2E == RdW) & RegwriteM | 2'b01           |

Hardware required for forwarding unit- 2 Mux before ALU and Hazard Unit

Hazard Unit inputs are Rs1E, Rs2E (Execute stage sources) , RdM, RdW (destinations) , RegWriteM, RegWriteW

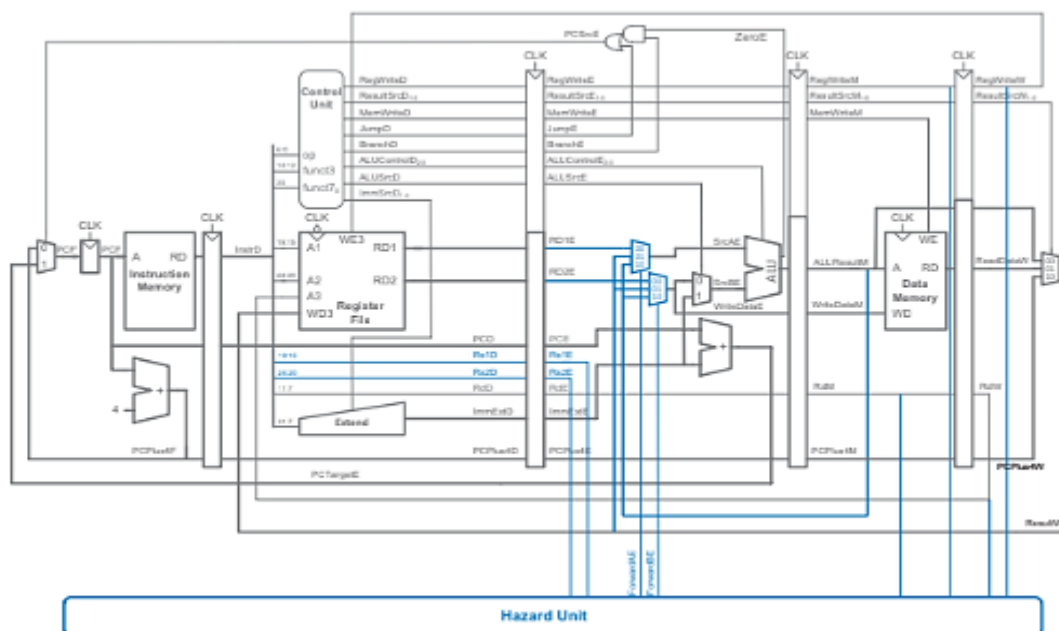


Figure 18 Pipeline Processor to solve Data Hazards with Forwarding

## 2) Load use Data Hazard (forwarding fails)

Problem:

lw s7, 0(s1) # load from Data memory  
and t0, s7, s2 # needs s7 immediately

Lw gets data at the end of MEM stage. and instruction needs it at start of EX stage. So, data is not ready in time. Therefore, Forwarding cannot fix this issue.

### Solution: Stall pipeline

Insert bubbles (nop) it freezes pipeline temporarily until data is available. shows stalling the dependent instruction (and) in the Decode stage. and enters the Decode stage in cycle 3 and stalls there through cycle 4. The subsequent instruction (or) must remain in the Fetch stage during both cycles as well because the Decode stage is full. In cycle 5, the result can be forwarded from the Writeback stage of lw to the Execute stage of and. Also, in cycle 5, source s7 of the or instruction is read directly from the register file, with no need for forwarding. Note that the Execute stage is unused in cycle 4. Likewise, Memory is unused in cycle 5 and Writeback is unused in cycle 6. This unused stage propagating through the pipeline is called a bubble, which behaves like a nop instruction. The bubble is introduced by zeroing out the Execute stage control signals during a Decode stage stall so that the bubble performs no action and changes no architectural state. In summary, stalling a stage is performed by disabling its pipeline register (i.e., the register to the left of a stage) so that the stage's inputs do not change. When a stage is stalled, all previous stages must also be stalled so that no subsequent instructions are lost. The pipeline register directly after the stalled stage must be cleared (flushed) to prevent bogus information from propagating forward. Stalls degrade performance, so they should be used only when necessary.

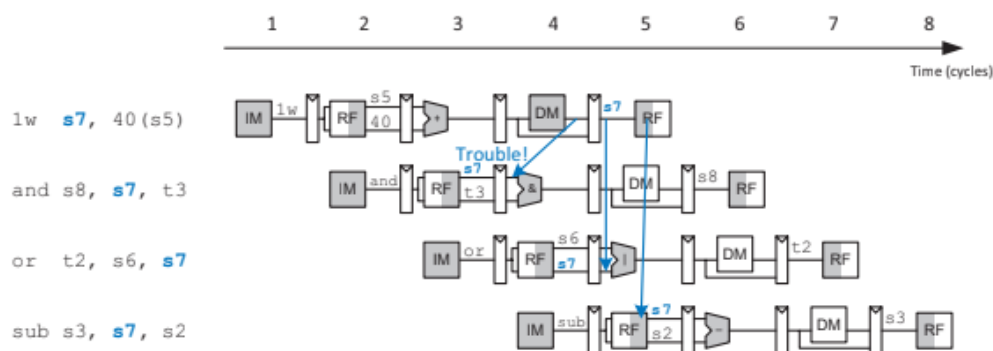


Figure 19 Pipeline illustration trouble forwarding from lw

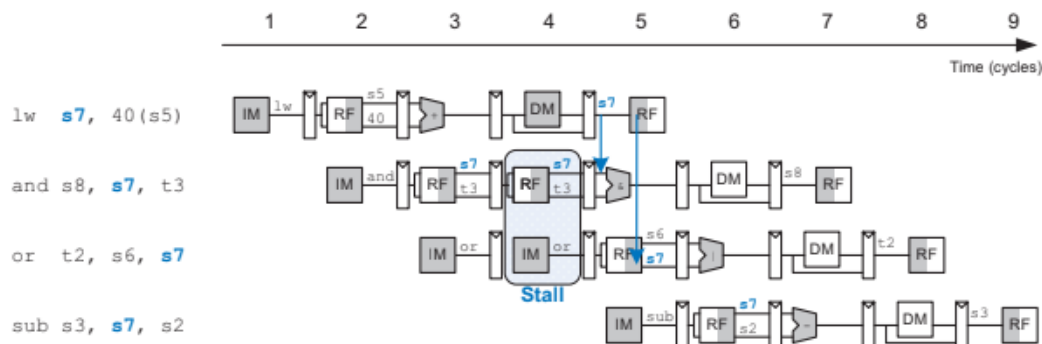
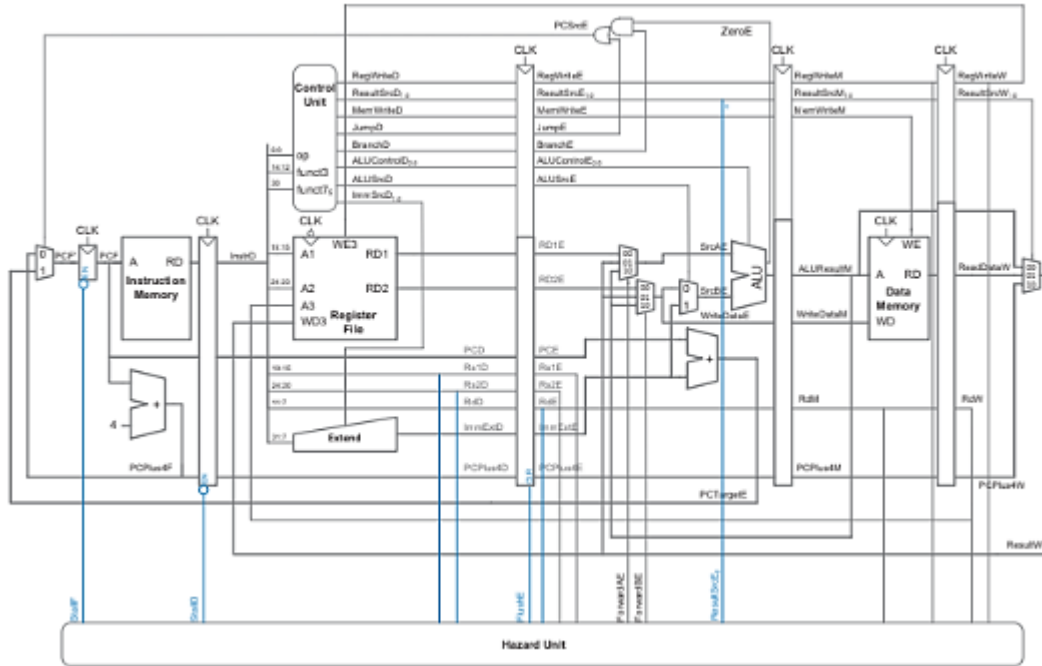


Figure 20 Pipeline illustrating stalls to solve Data Hazards



**Figure 21 Pipeline Processor with Stalls to solve lw data Hazard**

modifies the pipelined processor to add stalls for lw dependencies. In order for the Hazard Unit to stall the pipeline, the following conditions must be met:

1. A load word is in the Execute stage (indicated by ResultSrcE0 = 1) and
  2. The load's destination register (RdE) matches Rs1D or Rs2D, the source operands of the instruction in the Decode stage
- Stalls are supported by adding enable inputs (EN) to the Fetch and Decode pipeline registers and a synchronous reset/clear (CLR) input to the Execute pipeline register. When a load word (lw) stall occurs, StallID and StallF are asserted to force the Decode and Fetch stage pipeline registers to retain their existing values. FlushE is also asserted to clear the contents of the Execute stage pipeline register, introducing a bubble. The Hazard Unit lwStall (load word stall) signal indicates when the pipeline should be stalled due to a load word dependency. Whenever lwStall is TRUE, all of the stall and flush signals are asserted. Hence, the logic to compute the stalls and flushes is:

$$\text{lwStall} = \text{ResultSrcE0} \ \& \ ((\text{Rs1D} == \text{RdE}) \mid (\text{Rs2D} == \text{RdE}))$$

$$\text{StallF} = \text{StallID} = \text{FlushE} = \text{lwStall}$$

### 3) Control Hazard Problem and its Solution

#### Problem:

```

beq s1, s2, target

add t0, t1, t2 # may be wrong

sub t3, t4, t5 # may be wrong

target:

or t6, t7, t8
    
```

CPU doesn't know branch result yet.

It continues fetching next instructions.

If branch is taken → wrong instructions executed.

| <b>Solution1 : Stalling</b>                                                                                                                                                                                                                                 | <b>Solution2: Predict + flush</b>                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>beq s1, s2, target</b><br><br><b>nop</b><br><br><b>nop</b>                                                                                                                                                                                               | <b>beq s1, s2, target</b><br><br><b>add t0, t1, t2</b><br><br><b>sub t3, t4, t5</b>                                                                                                                                                                                                                                                                                                                                        |
| <p>stall the pipeline means pause instruction fetch until the branch condition is evaluated (i.e., PCSrcE is computed).</p> <p>Because the decision is made in the Execute stage, the pipeline would have to be stalled for two cycles at every branch.</p> | <p>a) If branch is taken:</p> <p>CPU begin executing add and sub next instructions are flushed (discarded) from decode and Execute stages.</p> <p>These wasted instruction cycles are called the branch misprediction penalty.</p> <p>b) the processor predicts that branches are not taken and simply continues executing the program in order until PCSrcE is asserted to select the next PC from PCTargetE instead.</p> |
| Wait until branch resolves (slow )                                                                                                                                                                                                                          | Faster than Stalling but small penalty                                                                                                                                                                                                                                                                                                                                                                                     |



Take this program for understanding:

**beq s1, s2, target**

**add t0, t1, t2**

**sub t3, t4, t5**

**target:**

**or t6, t7, t8**

| Branch not Taken                       | Branch Taken                                                                                                       |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Condition is false ( $s1 \neq s2$ )    | Condition is true ( $s1 = s2$ )                                                                                    |
| CPU continues sequentially             | CPU jumps to target                                                                                                |
| Execution order: beq-> add-> sub -> or | Execution order: beq-> target                                                                                      |
| No flushing needed                     | But the CPU already fetched: add , sub<br><br>Which are wrong instructions. Therefore must be discarded (flushed). |

In a pipeline:

- Fetch happens before branch decision is known
- Branch decision is resolved in Execute (EX) stage

So CPU must guess what to do next.

CPU Prediction “Branch is NOT TAKEN” . So, CPU does:

|                                      |
|--------------------------------------|
| beq s1, s2, target                   |
| add t0, t1, t2 # fetched immediately |
| sub t3, t4, t5 # fetched next        |

| CPU Prediction is correct (not taken)              | CPU Prediction is wrong ( branch taken)                                                                  |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Instructions already fetched (add, sub) are valid. | Cycle    Action<br>1        Fetch beq<br>2        Fetch add<br>3        Fetch sub + branch decision made |

|            |                                                                                  |
|------------|----------------------------------------------------------------------------------|
|            | 4 Realize branch should go to target                                             |
| No flush   | Flushing wrong instructions (add , sub) by clearing pipeline registers (bubbles) |
| No penalty | 2 cycles are lost so penalty                                                     |

When a branch is taken (indicated by PCSrcE being 1), FlushD and FlushE (output signals of HAZARD unit) must be asserted to flush the Decode and Execute pipeline registers. The flushes are now calculated as:

$$\text{FlushD} = \text{PCSrcE}$$

$$\text{FlushE} = \text{lwStall} \mid \text{PCSrcE}$$

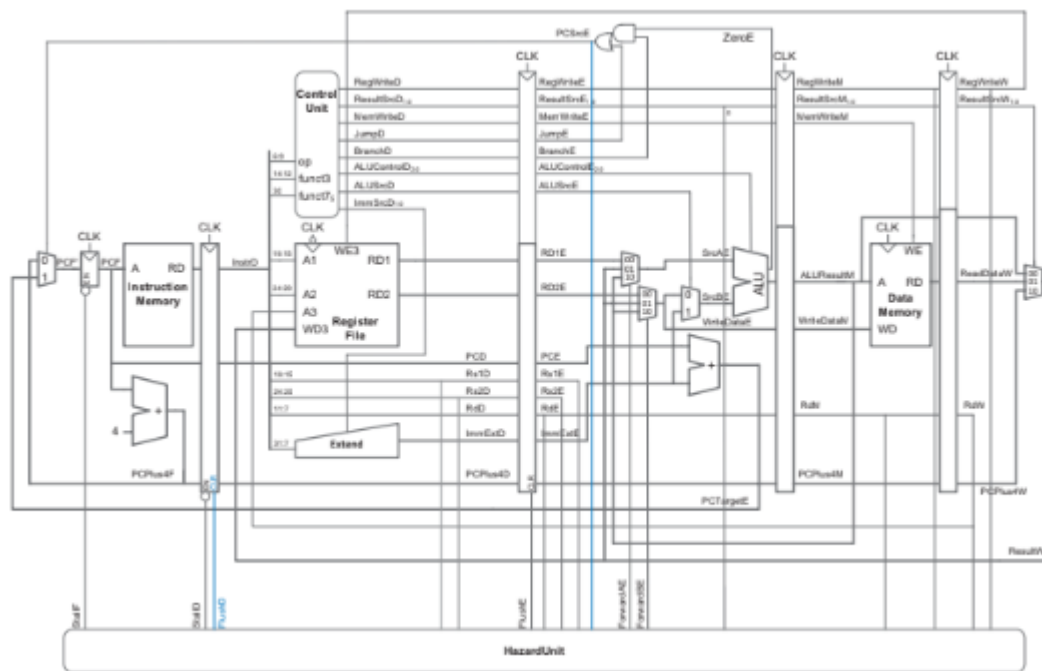


Figure 22 hazard unit for handling control hazard